Name: Dazon, Joshua Aareon Danielle P.	**Date Performed: Jan 8, 2026**
Course/Section: CPE 212-CPE31S1	**Date Submitted: Jan 8, 2026**
Instructor: Engr. Robin Valenzuela	**Semester and SY: 2nd Sem SY 25-26**

Activity 4: Running Elevated Ad hoc Commands

1. **Objectives:**

1.1 Use commands that makes changes to remote machines

1.2 Use playbook in automating ansible commands

2. **Discussion:**

Provide screenshots for each task.

Elevated Ad hoc commands

So far, we have not performed ansible commands that makes changes to the remote servers. We manage to gather facts and connect to the remote machines, but we still did not make changes on those machines. In this activity, we will learn to use commands that would install, update, and upgrade packages in the remote machines. We will also create a playbook that will be used for automations.

Playbooks record and execute **Ansible**'s configuration, deployment, and orchestration functions. They can describe a policy you want your remote systems to enforce, or a set of steps in a general IT process. If Ansible modules are the tools in your workshop, playbooks are your instruction manuals, and your inventory of hosts are your raw material. At a basic level, playbooks can be used to manage configurations of and deployments to remote machines. At a more advanced level, they can sequence multi-tier rollouts involving rolling updates, and can delegate actions to other hosts, interacting with monitoring servers and load balancers along the way. You can check this documentation if you want to learn more about playbooks. Working with playbooks — Ansible Documentation

Task 1: Run elevated ad hoc commands

1. Locally, we use the command *sudo apt update* when we want to download package information from all configured resources. The sources often defined in /etc/apt/sources.list file and other files located in /etc/apt/sources.list.d/ directory. So, when you run update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run

an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

What is the result of the command? Is it successful?

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible all -m apt -a update cache=true
    },
    "changed": false,
    "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /
var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13:
 Permission denied)"
}
```

- **The command fails since it needs permission.**

Try editing the command and add something that would elevate the privilege. Issue the command *ansible all -m apt -a update_cache=true --become --ask-become-pass.* Enter the sudo password when prompted. You will notice now that the output of this command is a success. The *update_cache=true* is the same thing as running *sudo apt update*. The --become command elevate the privileges and the *--ask-become-pass* asks for the password. For now, even if we only have changed the packaged index, we were able to change something on the remote server.

You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible all -m apt -a update_cache=true -
-become --ask-become-pass
BECOME password:
server1 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767863730,
    "cache_updated": true,
    "changed": true
}
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: *ansible all -m apt -a name=vim-nox --become --ask-become-pass.* The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible all -m apt -a name=vim-nox --becc
me --ask-become-pass
m) in auto mode",
        "update-alternatives: using /usr/bin/vim.nox to provide /usr/bin/vi (vi) i
n auto mode",
        "update-alternatives: using /usr/bin/vim.nox to provide /usr/bin/view (vie
w) in auto mode",
        "update-alternatives: using /usr/bin/vim.nox to provide /usr/bin/vim (vim)
 in auto mode",
        "update-alternatives: using /usr/bin/vim.nox to provide /usr/bin/vimdiff (
vimdiff) in auto mode",
        "Processing triggers for fontconfig (2.15.0-1.1ubuntu2) ...",
        "Processing triggers for libc-bin (2.39-0ubuntu8.6) ...",
        "Processing triggers for man-db (2.12.0-4build2) ..."
    ]
}
```

2.1 Verify that you have installed the package in the remote servers. Issue the command *which vim* and the command *apt search vim-nox* respectively. Was the command successful?

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble-updates,noble-security 2:9.1.0016-1ubuntu7.9 amd64
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.9 amd64 [installed,a
utomatic]
  Vi IMproved - enhanced vi editor - compact version
```

- **Yes, the command is successful.**

2.2 Check the logs in the servers using the following commands: *cd /var/log*. After this, issue the command *ls,* go to the folder *apt* and open history.log. Describe what you see in the history.log.

- In history.log it records where I update the ansible.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

3.1 Issue the command: *ansible all -m apt -a name=snapd --become --ask-become-pass*

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible all -m apt -a name=snapd --become
 --ask-become-pass
BECOME password:
server1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767863730,
    "cache_updated": false,
    "changed": false
}
```

- **The command is successfully executed.**

3.2 Now, try to issue this command: *ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass*
Describe the output of this command. Notice how we added the command *state=latest* and placed them in double quotations.

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible all -m apt -a "name=snapd state=l
atest" --become --ask-become-pass
BECOME password:
server1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1767863730,
    "cache_updated": false,
    "changed": false
}
```

4. At this point, make sure to commit all changes to GitHub.

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (*CPE232_yourname*). Issue the command *nano install_apache.yml*. This will create a playbook file called *install_apache.yml*. The .yml is the basic standard extension for playbook files.

When the editor appears, type the following:

```
  GNU nano 4.8                    install_apache.yml
---
- hosts: all
  become: true
  tasks:

  - name: install apache2 package
    apt:
      name: apache2
```

```
- hosts: web_servers
  gather_facts: yes

  tasks:
    - name: Install apache2 package
      apt:
        name: apache2
```

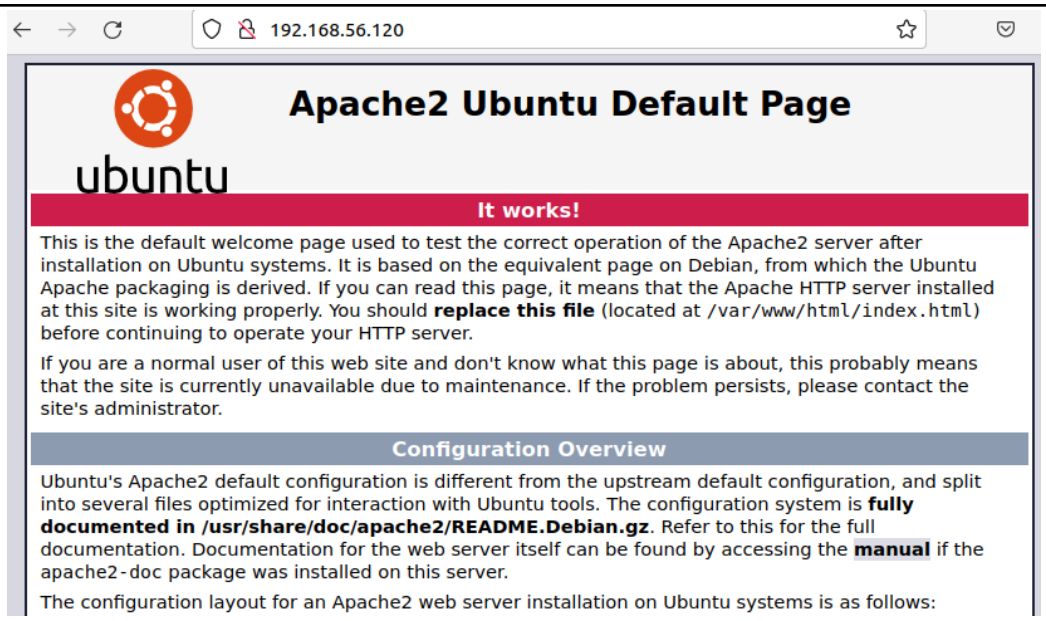
Make sure to save the file. Take note also of the alignments of the texts.

2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml.* Describe the result of this command.

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible-playbook --ask-become-pass ./inst
all_apache.yml
BECOME password:

PLAY [web_servers] *************************************************************

TASK [Gathering Facts] *********************************************************
ok: [server1]

TASK [Install apache2 package] *************************************************
ok: [server1]

PLAY RECAP *********************************************************************
server1                    : ok=2    changed=0    unreachable=0    failed=0    ski
pped=0    rescued=0    ignored=0
```

- **The automatically install apache packa**

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.



4. Try to edit the *install_apache.yml* and change the name of the package to any name that will not be recognized. What is the output?
5. This time, we are going to put additional task to our playbook. Edit the *install_apache.yml*. As you can see, we are now adding an additional command, which is the *update_cache*. This command updates existing package-indexes on a supportingname distro but not upgrading installed-packages (utilities) that were being installed.

```
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2
```

Save the changes to this file and exit.

```
nstall_apache.yml
- name: Install_ apache package
  hosts: web_servers
  gather_facts: yes

  tasks:
  - name: update repo index
    apt:
      update_cache: yes

  - name: Install apache2 package
    apt:
      name: apache2
```

6. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible-playbook --ask-become-pass ./inst
all apache.yml
PLAY RECAP *********************************************************************
server1                    : ok=2    changed=0    unreachable=0    failed=0    ski
pped=0    rescued=0    ignored=0

joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible-playbook --ask-become-pass ./inst
all_apache.yml
BECOME password:

PLAY [Install_ apache package] *************************************************

TASK [Gathering Facts] *********************************************************
ok: [server1]

TASK [update repo index] *******************************************************
```

```
ok: [server1]

TASK [update repo index] *******************************************************
changed: [server1]
```

- **Yes, there are two new changes to server 1**

7. Edit again the *install_apache.yml*. This time, we are going to add a PHP support for the apache package we installed earlier.

```yaml
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
```

Save the changes to this file and exit.

```yaml
- name: Install_ apache package
  hosts: web_servers
  gather_facts: yes

  tasks:
  - name: update repo index
    apt:
      update_cache: yes

  - name: Install apache2 package
    apt:
      name: apache2

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
```

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
joshua-dazon@LocalMachine:~/CPE212_31S1$ ansible-playbook --become --ask-become-pa
ss ./install apache.yml

PLAY [Install_ apache package] *************************************************

TASK [Gathering Facts] *********************************************************
ok: [server1]

TASK [update repo index] *******************************************************
changed: [server1]

TASK [Install apache2 package] *************************************************
ok: [server1]

TASK [add PHP support for apache] **********************************************
changed: [server1]

PLAY RECAP *********************************************************************
server1                    : ok=4    changed=2    unreachable=0    failed=0    ski
pped=0    rescued=0    ignored=0
```

- **Yes, there is new 2 recent changes in the servers**

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

```
joshua-dazon@LocalMachine:~/CPE232_joshua$ git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 945 bytes | 945.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:DazonJoshua/CPE232_joshua.git
   b0bc7fc..1cbf0ce  main -> main
branch 'main' set up to track 'origin/main'.
```

Reflections:

Answer the following:

1. What is the importance of using a playbook?
   - It acts as blueprint for automating task to another machine.
2. Summarize what we have done on this activity.
   - In this activity we learn to create an ansible playbook in yml. We install the apache package by automating the playbook.